Playtest Feedback Platform for Indie Developers

CS146S Final Project Proposal | Paprika Chen & Amilia Liu | October 1, 2026

We will build a web platform for indie developers to upload games and collect feedback linked to gameplay recordings. After playing, testers receive optional AI reminders based on their own comments to help complete a questionnaire.

Problem Statement

Some indie developers prefer observing playtests at exhibitions because players express confusion and suggestions as they play. Online questionnaires can lose these immediate reactions:

- **Players forget details.** By the end of a demo, they may forget a confusing moment or skip explaining it because recalling it takes effort.
- **Developers lack context.** A comment such as “this part is difficult” may not identify where or why the problem occurred.

We will capture comments with their video context and use AI to help players recall them afterward. Interviews and pilots will test whether this produces more useful feedback.

Target Users and Business Context

- **Developers:** Indie developers and small teams with playable web prototypes.
- **Testers:** Friends, community members, and fellow developers invited by the game’s creator.
- **Initial service:** Game sharing, feedback capture, questionnaires, and review. Recruitment and pricing will be considered after demand validation.
- **Product focus:** *PlaytestCloud* and *Antidote* already offer recording and voice feedback. We will evaluate connecting in-play comments to questionnaire reminders, with answers traceable to the original comments and footage.
- **Early validation:** We plan to contact a game development professor and classmates, and seek indie developer interviews. Classroom use is a possible later application; adoption is not confirmed.

Core Features

- **Upload and share.** Upload a web-build ZIP, configure a questionnaire, and create a playtest link. Establish one supported engine export configuration through an early prototype.
- **Comment during play.** After authorizing recording, testers type feedback or hold a key to record voice comments. Comments link to footage before and after the moment.
- **Complete a questionnaire.** AI matches transcribed or typed comments to questions and presents the original wording and timestamps. Players can view footage, add or edit a reminder, or dismiss it. Nothing is filled automatically.
- **Review feedback.** Developers browse answers, comments, and recordings by project, build, and session, and jump to relevant footage.

Scope and Implementation

MVP: Browser playtesting, full-session recording with timestamps, questionnaires, speech transcription, and AI recall prompts. No game-event SDK is required.

System: Project management, isolated game hosting, media storage, feedback services, asynchronous AI processing, and a review interface. Shared IDs link builds, sessions, comments, and answers.

Reliability: Validate uploaded packages, upload recordings in segments, show recording and upload status, and keep questionnaires usable when AI processing fails. Restrict feedback access and allow recording deletion.

Stretch goals: An event SDK, heatmaps, and detailed version comparisons. Desktop executables, a level editor, a tester marketplace, and class management are outside the MVP.

Validation and Success Criteria

User testing: Interview developers and run real-game pilots. Observe recording performance, feedback-panel interference, and willingness to comment.

Functional targets: Complete the upload-to-review workflow for supported games; align comments and video within one second; preserve each feedback item's source.

AI quality: Use human-labeled examples to check source attribution, preserved meaning, and question relevance. Neutral comments must not become negative judgments.

Questionnaire evaluation: Compare completion time and feedback specificity with and without reminders, and ask whether prompts steer answers. Longer answers alone do not establish success; small pilots do not establish general effectiveness.

AI Development Plan

Context and UI: Give coding assistants the build format, session timeline, data schema, and acceptance criteria. Use AI to prototype the feedback panel and reminder UI; validate them with testers.

Implementation: Use AI for ZIP validation, hosting, recording uploads, and comment-to-video timing. Verify generated code with real builds, interrupted uploads, and stopped screen sharing.

Product AI: Integrate transcription and develop comment-to-question prompts that preserve source IDs and timestamps. Check outputs against labeled examples and retain player control of answers.

Tests and review: Use AI to propose tests for timing drift, denied microphone access, empty transcripts, duplicate uploads, invalid packages, and cross-project access. Review assertions, measure coverage, and have teammates verify AI review findings about cleanup, retries, permissions, and output validation.

CI and deployment: Use AI to configure automated checks and game/media serving settings. Validate the deployed upload-to-feedback workflow.

Evidence and teamwork: Use Git, peer reviews, and a project board. Keep each AI task's requirement, actual prompt, code change, validation result, and human correction. Document responsibilities, prompt revisions, failures, and lessons learned.